

Laboration – Dataintegration och API-kontrakt

Syfte

I den här laborationen ska vi gå från en enkel simulering till ett riktigt integrationsflöde. Vi kommer att använda **OpenAPI** för att definiera vårt kontrakt och **SQLite** för att spara data persistent.

Du kommer att:

- Läsa och förstå en OpenAPI-specifikation.
- Uppdatera ett Flask-API för att använda en databas (SQLite).
- Bygga en mock-sensor som följer ett strikt API-kontrakt.
- Skapa en vy för att se den historiska datan.

Förberedelser

Vi utgår från projektet `iot-api` som vi skapade i förra lektionen. Om du inte har det kvar, skapa en ny mapp och installera Flask:

```
mkdir iot-integration  
cd iot-integration  
pip install flask requests
```

Kopiera filen `openapi.yaml` från lektionsmappen till din projektmappp så att du har den som referens.

Del 1 – Förstå kontraktet

Öppna `openapi.yaml`. Detta är vår "ritning".

1. Vilka endpoints finns definierade?
2. Vilka fält *måste* skickas med när vi postar data till `/sensor-data` ?
3. Vilket format förväntas temperaturen vara i?

Del 2 – Persistent lagring med SQLite

Vi ska nu ändra vårt API så att det inte tappas all data när vi startar om servern. Vi använder `sqlite3` som ingår i Python.

Skapa `database.py`

Skapa en hjälpfil för att hantera databasen:

```
import sqlite3

def init_db():
    conn = sqlite3.connect('sensor_data.db')
    c = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS readings (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            device TEXT NOT NULL,
            temp REAL NOT NULL,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    ''')
    conn.commit()
    conn.close()

def save_reading(device, temp):
    conn = sqlite3.connect('sensor_data.db')
    c = conn.cursor()
```

Del 3 – Mocka sensorn enligt kontraktet

Nu ska vi bygga en sensor-mock som följer specifikationen i `openapi.yaml`.

Skapa `mock_sensor.py`

```
import requests
import time
import random

# Inställningar
API_URL = "http://localhost:5000/sensor-data"
DEVICE_ID = "sensor-mock-01"

def run_sensor():
    print(f"Startar mock-sensor: {DEVICE_ID}")
    while True:
        # Generera data enligt kontraktet
        payload = {
            "device": DEVICE_ID,
            "temp": round(random.uniform(18.0, 26.0), 2)
        }

        try:
            response = requests.post(API_URL, json=payload)
            if response.status_code == 200:
                print(f"Skickat: {payload['temp']}°C - Status: {response.json()['status']}")
            else:
                print(f"Fel vid sändning: {response.status_code}")
        except Exception as e:
            print(f"Exception: {e}")
```

Del 4 – Validering och Test

1. Starta API:et: `python app.py`
2. Starta mock-sensorn i ett annat terminalfönster: `python mock_sensor.py`
3. Låt den gå i någon minut.
4. Besök `http://localhost:5000/temperature` i din webbläsare.
5. **Verifiera:** Stoppa API-servern (Ctrl+C) och starta den igen. Ligger datan kvar?

Fördjupningsuppgifter

Om du har tid över, prova följande:

1. **Schema-validering:** Försök skicka data från `mock_sensor.py` som *bryter* mot kontraktet (t.ex. ändra `temp` till en sträng eller ta bort `device`). Hur reagerar API:et?
2. **Visualisering:** Skapa en enkel HTML-sida som hämtar datan från `/temperature` och visar den i en tabell eller med ett diagram (t.ex. Chart.js).
3. **Flera sensorer:** Starta flera instanser av `mock_sensor.py` med olika `DEVICE_ID` och se hur databasen hanterar dem.
4. **Tidsfiltrering:** Uppdatera API:et så att man kan skicka med en parameter för att bara se data från den senaste timmen.

Sammanfattning

Du har nu byggt ett komplett, persistent integrationsflöde. Genom att använda ett API-kontrakt (OpenAPI) har du säkerställt att sensorn och API:et pratar samma språk, och genom att använda en databas har du skapat ett system som kan hantera historisk data över tid.